

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib.ticker as ticker
import matplotlib.cm as cm
from warnings import filterwarnings
filterwarnings('ignore')
```

```
In [3]: # set professional color theme
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette("viridis")
viridis_colors=cm.viridis(np.linspace(0,1,5))
primary_colors=viridis_colors[0]
secondary_colors=viridis_colors[1]
accent_colors=viridis_colors[2]
danger_colors='#800000'
neutral_colors=viridis_colors[4]
custom_palette=viridis_colors
```

```
In [4]: df=pd.read_csv("DataCoSupplyChainDataset.csv",encoding='latin-1')
```

```
In [5]: df
```

Out[5]:

	Type	Days for shipping (real)	Days for shipment (scheduled)	Benefit per order	Sales per customer	Delivery Status	Late_delivery
0	DEBIT	3	4	91.250000	314.640015	Advance shipping	
1	TRANSFER	5	4	-249.089996	311.359985	Late delivery	
2	CASH	4	4	-247.779999	309.720001	Shipping on time	
3	DEBIT	3	4	22.860001	304.809998	Advance shipping	
4	PAYMENT	2	4	134.210007	298.250000	Advance shipping	
...	...	...	...	...	...	...	...
180514	CASH	4	4	40.000000	399.980011	Shipping on time	
180515	DEBIT	3	2	-613.770019	395.980011	Late delivery	
180516	TRANSFER	5	4	141.110001	391.980011	Late delivery	
180517	PAYMENT	3	4	186.229996	387.980011	Advance shipping	
180518	PAYMENT	4	4	168.949997	383.980011	Shipping on time	

180519 rows × 53 columns



In [6]: df.shape

Out[6]: (180519, 53)

In [7]: df.columns

```
Out[7]: Index(['Type', 'Days for shipping (real)', 'Days for shipment (scheduled)',  
             'Benefit per order', 'Sales per customer', 'Delivery Status',  
             'Late_delivery_risk', 'Category Id', 'Category Name', 'Customer City',  
             'Customer Country', 'Customer Email', 'Customer Fname', 'Customer Id',  
             'Customer Lname', 'Customer Password', 'Customer Segment',  
             'Customer State', 'Customer Street', 'Customer Zipcode',  
             'Department Id', 'Department Name', 'Latitude', 'Longitude', 'Market',  
             'Order City', 'Order Country', 'Order Customer Id',  
             'order date (DateOrders)', 'Order Id', 'Order Item Cardprod Id',  
             'Order Item Discount', 'Order Item Discount Rate', 'Order Item Id',  
             'Order Item Product Price', 'Order Item Profit Ratio',  
             'Order Item Quantity', 'Sales', 'Order Item Total',  
             'Order Profit Per Order', 'Order Region', 'Order State', 'Order Status',  
             'Order Zipcode', 'Product Card Id', 'Product Category Id',  
             'Product Description', 'Product Image', 'Product Name', 'Product Price',  
             'Product Status', 'shipping date (DateOrders)', 'Shipping Mode'],  
            dtype='str')
```

exploratory data analysis(EDA)

```
In [8]: # overview  
print('row ,cols:',df.shape)  
print('\nncolumns:')  
print(df.columns.tolist())  
print('\nNum duplicates:',df.duplicated().sum())  
print('\nMissingvalues:(top 20):')  
print(df.isna().sum().sort_values(ascending=False).head(20))
```

```
row ,cols: (180519, 53)
```

```
columns:
```

```
['Type', 'Days for shipping (real)', 'Days for shipment (scheduled)', 'Benefit pe  
r order', 'Sales per customer', 'Delivery Status', 'Late_delivery_risk', 'Categor  
y Id', 'Category Name', 'Customer City', 'Customer Country', 'Customer Email', 'C  
ustomer Fname', 'Customer Id', 'Customer Lname', 'Customer Password', 'Customer S  
egment', 'Customer State', 'Customer Street', 'Customer Zipcode', 'Department I  
d', 'Department Name', 'Latitude', 'Longitude', 'Market', 'Order City', 'Order Co  
untry', 'Order Customer Id', 'order date (DateOrders)', 'Order Id', 'Order Item C  
ardprod Id', 'Order Item Discount', 'Order Item Discount Rate', 'Order Item Id',  
'Order Item Product Price', 'Order Item Profit Ratio', 'Order Item Quantity', 'Sa  
les', 'Order Item Total', 'Order Profit Per Order', 'Order Region', 'Order Stat  
e', 'Order Status', 'Order Zipcode', 'Product Card Id', 'Product Category Id', 'P  
roduct Description', 'Product Image', 'Product Name', 'Product Price', 'Product S  
tatus', 'shipping date (DateOrders)', 'Shipping Mode']
```

```
Num duplicates: 0
```

```
Missingvalues:(top 20):
```

Product Description	180519
Order Zipcode	155679
Customer Lname	8
Customer Zipcode	3
Days for shipment (scheduled)	0
Sales per customer	0
Benefit per order	0
Delivery Status	0
Late_delivery_risk	0
Customer City	0
Customer Country	0
Category Id	0
Category Name	0
Customer Fname	0
Customer Email	0
Customer Password	0
Customer Id	0
Customer Segment	0
Customer State	0
Days for shipping (real)	0

```
dtype: int64
```

```
In [9]: df['Benefit per order']==df['Order Profit Per Order']
```

```
Out[9]: 0      True
1      True
2      True
3      True
4      True
...
180514  True
180515  True
180516  True
180517  True
180518  True
Length: 180519, dtype: bool
```

```
In [10]: (df['Benefit per order']==df['Order Profit Per Order']).value_counts()
```

```
Out[10]: True      180519
         Name: count, dtype: int64
```

```
In [11]: # data cleaning
columns_to_drop = [
    'Product Description',
    'Product Image',
    'Customer Email',
    'Customer Fname',
    'Customer Lname',
    'Customer Password',
    'Customer Street',
    'Customer Zipcode',
    'Order Zipcode',
    'Latitude',
    'Longitude',
    'Order Item Cardprod Id',
    'Order Item Discount',
    'Order Item Discount Rate',
    'Order Item Id',
    'Order Item Product Price',
    'Order Item Quantity',
    'Order Item Total',
    'Category Id',
    'Department Id',
    'Order Id',
    'Order Customer Id',
    'Product Card Id',
    'Product Category Id',
    'Benefit per order',
    'Product Status',
    'Customer City',
    'Order City',
    'Customer State',
    'Order State',
    'Market',
    'Customer Id', 'Order Country'
]

df = df.drop(columns=columns_to_drop, errors='ignore')
```

```
In [12]: df=df[df['Delivery Status'] != 'Shipping canceled']
```

```
In [13]: df.columns = df.columns.str.strip()
```

```
In [14]: print([repr(col) for col in df.columns if 'date' in col.lower()])

['order date (DateOrders)', 'shipping date (DateOrders)']
```

```
In [15]: for c in ['order date (DateOrders)', 'shipping date (DateOrders)']:
         df[c] = pd.to_datetime(df[c], errors='coerce', dayfirst=False)
```

```
In [16]: print('row ,cols:',df.shape)
         print('\nMissing values:(top 5):')
         print(df.isna().sum().sort_values(ascending=False).head(5))
```

```
row ,cols: (172765, 20)
```

```
Missingvalues:(top 5):
```

```
Type                                0
Days for shipping (real)            0
Days for shipment (scheduled)       0
Sales per customer                  0
Delivery Status                     0
dtype: int64
```

```
In [17]: df.head()
```

```
Out[17]:
```

	Type	Days for shipping (real)	Days for shipment (scheduled)	Sales per customer	Delivery Status	Late_delivery_risk	Category Name
0	DEBIT	3	4	314.640015	Advance shipping	0	Sporting Goods
1	TRANSFER	5	4	311.359985	Late delivery	1	Sporting Goods
2	CASH	4	4	309.720001	Shipping on time	0	Sporting Goods
3	DEBIT	3	4	304.809998	Advance shipping	0	Sporting Goods
4	PAYMENT	2	4	298.250000	Advance shipping	0	Sporting Goods

```
In [18]: for col in df.columns:
          if df[col].nunique()<10:
              print(f'\n{col} value counts:')
              print(df[col].value_counts())
```

Type value counts:

Type

DEBIT	69295
TRANSFER	42129
PAYMENT	41725
CASH	19616

Name: count, dtype: int64

Days for shipping (real) value counts:

Days for shipping (real)

2	54205
6	27489
3	27478
4	27297
5	27003
0	4839
1	4454

Name: count, dtype: int64

Days for shipment (scheduled) value counts:

Days for shipment (scheduled)

4	103153
2	33806
1	26513
0	9293

Name: count, dtype: int64

Delivery Status value counts:

Delivery Status

Late delivery	98977
Advance shipping	41592
Shipping on time	32196

Name: count, dtype: int64

Late_delivery_risk value counts:

Late_delivery_risk

1	98977
0	73788

Name: count, dtype: int64

Customer Country value counts:

Customer Country

EE. UU.	106425
Puerto Rico	66340

Name: count, dtype: int64

Customer Segment value counts:

Customer Segment

Consumer	89420
Corporate	52528
Home Office	30817

Name: count, dtype: int64

Order Status value counts:

Order Status

COMPLETE	59491
PENDING_PAYMENT	39832
PROCESSING	21902
PENDING	20227
CLOSED	19616

```
ON_HOLD          9804
PAYMENT_REVIEW   1893
Name: count, dtype: int64
```

```
Shipping Mode value counts:
Shipping Mode
Standard Class    103153
Second Class      33806
First Class       26513
Same Day          9293
Name: count, dtype: int64
```

```
In [19]: df['Order_Processing_time']=(df['shipping date (DateOrders)']-df['order date (DateOrders)'])
df['Delay']=df['Order_Processing_time']-df['Days for shipment (scheduled)']
df['Is_Delayed']=df['Delay']>0
df['order_month']=df['order date (DateOrders)'].dt.month
df['order_day']=df['order date (DateOrders)'].dt.day_name()
df['order_hour']=df['order date (DateOrders)'].dt.hour
df.describe()
```

```
Out[19]:
```

	Days for shipping (real)	Days for shipment (scheduled)	Sales per customer	Late_delivery_risk	order date (DateOrders)
count	172765.000000	172765.000000	172765.000000	172765.000000	172765
mean	3.498596	2.933100	183.165948	0.572900	2016-06-12 15:25:39.457991
min	0.000000	0.000000	7.490000	0.000000	2015-01-01 00:00:00
25%	2.000000	2.000000	104.379997	0.000000	2015-09-21 18:01:00
50%	3.000000	4.000000	163.990005	1.000000	2016-06-11 08:11:00
75%	5.000000	4.000000	247.399994	1.000000	2017-02-28 21:08:00
max	6.000000	4.000000	1939.989990	1.000000	2018-01-31 23:38:00
std	1.623446	1.373405	120.141871	0.494659	NaN

```
In [20]: df['Is_Delayed'].value_counts()
```

```
Out[20]: Is_Delayed
True      94523
False     78242
Name: count, dtype: int64
```

```
In [21]: df.columns
```

```
Out[21]: Index(['Type', 'Days for shipping (real)', 'Days for shipment (scheduled)',
              'Sales per customer', 'Delivery Status', 'Late_delivery_risk',
              'Category Name', 'Customer Country', 'Customer Segment',
              'Department Name', 'order date (DateOrders)', 'Order Item Profit Ratio',
              'Sales', 'Order Profit Per Order', 'Order Region', 'Order Status',
              'Product Name', 'Product Price', 'shipping date (DateOrders)',
              'Shipping Mode', 'Order_Processing_time', 'Delay', 'Is_Delayed',
              'order_month', 'order_day', 'order_hour'],
              dtype='str')
```

```
In [22]: df['Order Profit Per Order']
```

```
Out[22]: 0          91.250000
         1        -249.089996
         2        -247.779999
         3          22.860001
         4         134.210007
         ...
        180514      40.000000
        180515     -613.770019
        180516      141.110001
        180517      186.229996
        180518      168.949997
        Name: Order Profit Per Order, Length: 172765, dtype: float64
```

```
In [23]: df['Order Profit Per Order'] > 0
```

```
Out[23]: 0          True
         1         False
         2         False
         3          True
         4          True
         ...
        180514       True
        180515       False
        180516       True
        180517       True
        180518       True
        Name: Order Profit Per Order, Length: 172765, dtype: bool
```

```
In [24]: df['Profitability Flag'] = np.where(df['Order Profit Per Order'] > 0, 'Profit', np.
        df['Profitability Flag'].value_counts())
```

```
Out[24]: Profitability Flag
Profit      139354
Loss        32295
Break-even   1116
Name: count, dtype: int64
```

```
In [25]: df['Profitability Flag'].value_counts(normalize=True)
```

```
Out[25]: Profitability Flag
Profit      0.80661
Loss        0.18693
Break-even   0.00646
Name: proportion, dtype: float64
```

```
In [26]: profit_counts = df['Profitability Flag'].value_counts(normalize=True) * 100

profit_counts.plot()
```

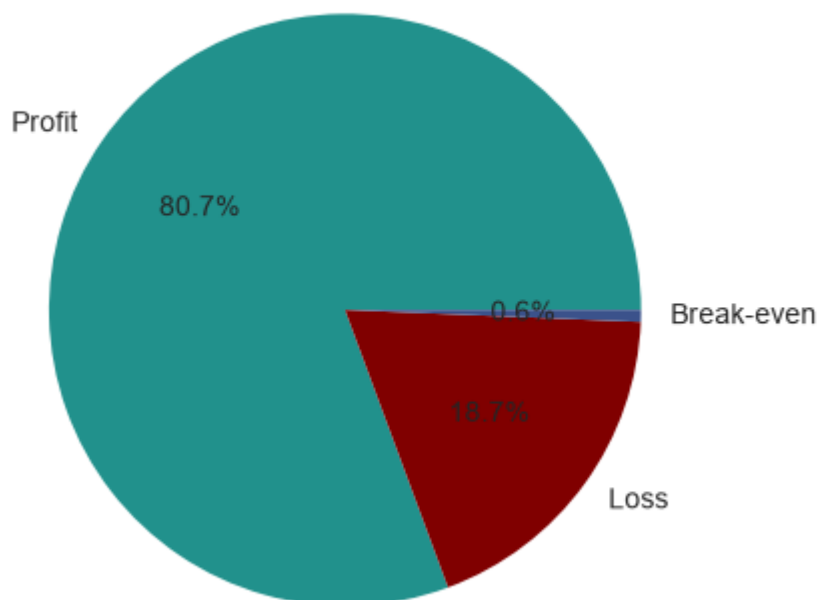
```

kind='pie',
autopct='%1.1f%%',
colors=[accent_colors, danger_colors, secondary_colors]
)

plt.ylabel('')
plt.title('Profitability Distribution (%)')
plt.show()

```

Profitability Distribution (%)



```

In [27]: def format_func(value):
            if value >= 1e6:
                return f'{value/1e6:.1f}M $'
            elif value >= 1e3:
                return f'{value:.0f} K $'
            else:
                return f'{value:.0f} $'

            delayed_df = df[df['Delay'] > 0]
            metrics = {}
            metrics['Total Orders'] = len(df)
            metrics['Late Deliveries'] = len(delayed_df)
            metrics['90% Delay(days)'] = delayed_df['Delay'].quantile(0.90)
            metrics['On time Delivery %'] = (1 - float(metrics['Late Deliveries'])) / metrics['Total Orders']
            metrics['Late Deliveries %'] = float(metrics['Late Deliveries']) / metrics['Total Orders']
            metrics['Total profit'] = format_func(df.loc[df['Order Profit Per Order'] > 0, 'Order Profit Per Order'].sum())
            metrics['Total Loss due to delays'] = format_func(df.loc[df['Delay'] > 0, 'Order Profit Per Order'].sum())

            print('\n---Business KPIs ---\n')

            for K, V in metrics.items():
                if isinstance(V, float):
                    print(f"{K}: {V:.2f}")
                else:
                    print(f"{K}: {V}")

```

---Business KPIs ---

Total Orders: 172765
Late Deliveries: 94523
90% Delay(days): 3.00
On time Delivery %: 45.29
Late Deliveries %: 54.71
Total profit: 7.5M \$
Total Loss due to delays: 2.1M \$

Profitability vs Delivery Time Analysis

```
In [28]: profit_metrics=(  
    df.groupby('Delay')['Order Profit Per Order']  
    .agg(  
        mean_profit='mean',  
        total_profit='sum',  
        order_count='count'  
    )  
    .reset_index()  
)
```

```
In [29]: profit_metrics
```

```
Out[29]:
```

	Delay	mean_profit	total_profit	order_count
0	-2	23.360134	4.875961e+05	20873
1	-1	21.604769	4.476292e+05	20719
2	0	22.249118	8.154302e+05	36650
3	1	22.333227	1.194895e+06	53503
4	2	21.128491	5.821110e+05	27551
5	3	20.031412	1.356527e+05	6772
6	4	21.368783	1.431067e+05	6697

```
In [30]: delay_distribution=(  
    df['Delay']  
    .value_counts(normalize=True)  
    .sort_index() * 100  
).reset_index()
```

```
In [31]: delay_distribution
```

Out[31]:

	Delay	proportion
0	-2	12.081730
1	-1	11.992591
2	0	21.213788
3	1	30.968657
4	2	15.947096
5	3	3.919775
6	4	3.876364

```

In [32]: delay_distribution.columns = ['Delay_Days', 'Percentage']

print("\nProfit Metrics by Delay Day:")
display(delay_distribution)

delay_distribution.columns = ['Delay_Days', 'Percentage']

print("\nProfit Metrics by Delay Day:")
display(delay_distribution)
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

sns.barplot(
    x='Delay_Days',
    y='Percentage',
    data=delay_distribution,
    color=accent_colors,
    ax=ax1)

ax1.set_title('Delay Distribution')
ax1.set_xlabel('Delay Days')
ax1.set_ylabel('Percentage of Orders (%)')
for bar in ax1.patches:
    height = bar.get_height()
    ax1.text(
        bar.get_x() + bar.get_width() / 2,
        height + 0.5,
        f'{height:.1f}%',
        ha='center',
        va='bottom')

ax2.set_ylabel("Total Profit", color=primary_colors)

ax2.bar(
    profit_metrics['Delay'],
    profit_metrics['total_profit'],
    color=primary_colors,
    label='Total Profit'
)

ax2.tick_params(axis='y', labelcolor=primary_colors)
ax2.set_xlabel("Delay Days")

```

```

ax3 = ax2.twinx()

ax3.set_ylabel("Mean Profit", color=accent_colors)

ax3.plot(
    profit_metrics['Delay'],
    profit_metrics['mean_profit'],
    marker='o',
    label='Mean Profit',
    color=accent_colors
)

ax3.tick_params(axis='y', labelcolor=accent_colors)
def format_func(value, tick_number):
    if value >= 1e6:
        return f'{value / 1e6:.1f}M $'
    elif value >= 1e3:
        return f'{value / 1e3:.0f}K $'
    else:
        return f'{value:.0f} $'

ax2.yaxis.set_major_formatter(
    ticker.FuncFormatter(format_func)
)

ax3.set_title("Profit Analysis for Delay Days")
lines, labels = ax3.get_legend_handles_labels()
lines2, labels2 = ax2.get_legend_handles_labels()

ax3.legend(
    lines + lines2,
    labels + labels2,
    loc='upper right'
)

ax3.grid(
    True,
    linestyle=':',
    alpha=0.5
)

plt.tight_layout()
plt.show()

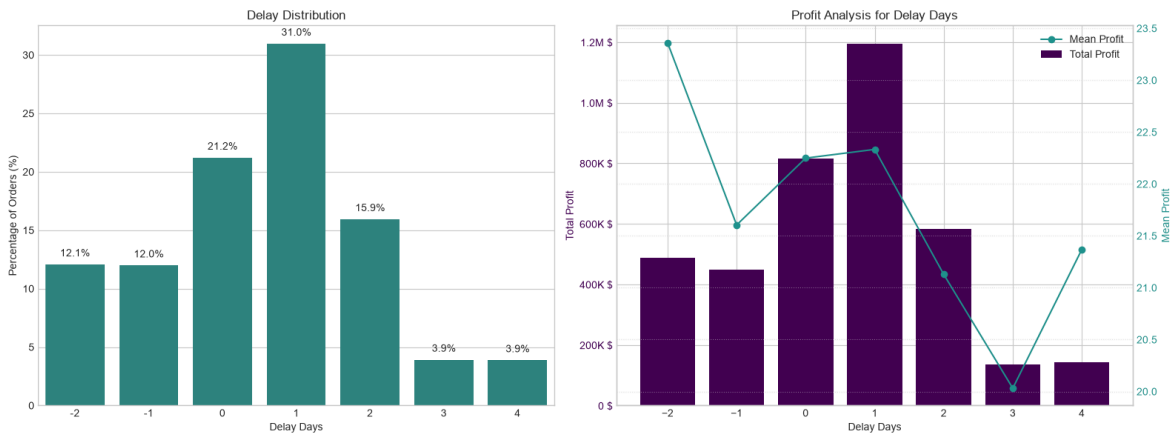
```

Profit Metrics by Delay Day:

	Delay_Days	Percentage
0	-2	12.081730
1	-1	11.992591
2	0	21.213788
3	1	30.968657
4	2	15.947096
5	3	3.919775
6	4	3.876364

Profit Metrics by Delay Day:

	Delay_Days	Percentage
0	-2	12.081730
1	-1	11.992591
2	0	21.213788
3	1	30.968657
4	2	15.947096
5	3	3.919775
6	4	3.876364



Bottleneck Detection

```
In [33]: import matplotlib.pyplot as plt
import seaborn as sns

def compute_delay_pct_by_category(category):
    cat_df = df.groupby(category).agg(
        total_orders=('Delay', 'count'),
        late_orders=('Is_Delayed', 'sum')
    ).reset_index()

    cat_df['delay_pct'] = (
        cat_df['late_orders'] / cat_df['total_orders'] * 100
    )
```

```

cat_df = cat_df.sort_values(
    'delay_pct',
    ascending=False
).head(10)

return cat_df

categories = [
    'Order Region',
    'Customer Segment',
    'Shipping Mode',
    'Order Status',
    'Type',
    'Department Name'
]

fig, axes = plt.subplots(
    2, 3,
    figsize=(16, 8),
    constrained_layout=True
)

axes = axes.flatten()

for ax, category in zip(axes, categories):

    cat_df = compute_delay_pct_by_category(category)

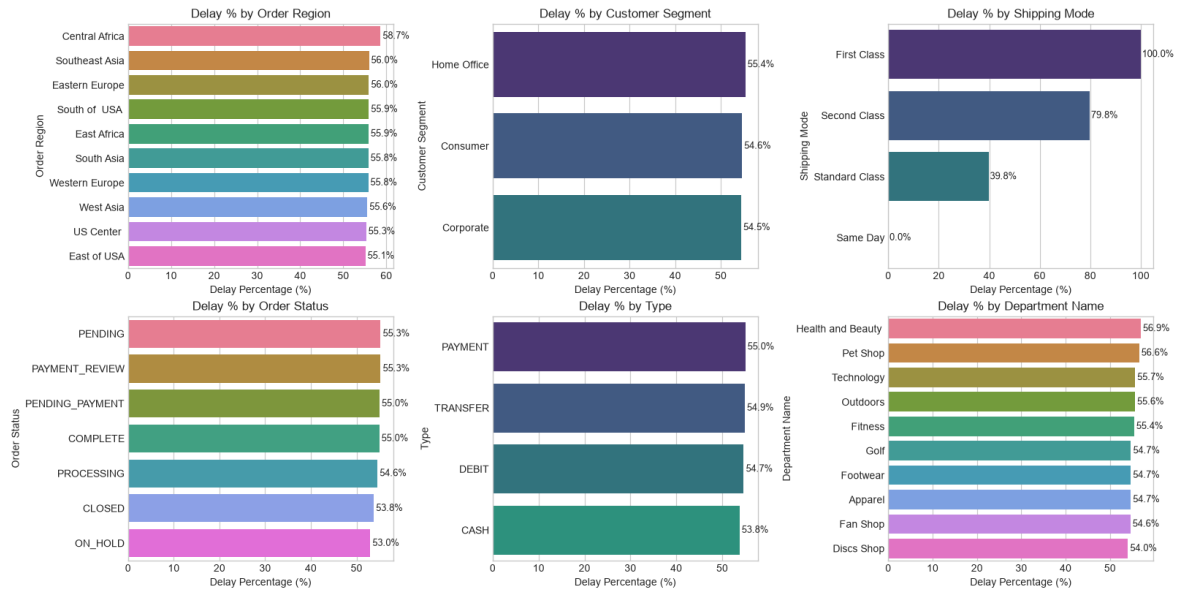
    sns.barplot(
        data=cat_df,
        x='delay_pct',
        y=category,
        ax=ax,
        hue=category,
        legend=False
    )

    ax.set_title(f'Delay % by {category}')
    ax.set_xlabel('Delay Percentage (%)')
    ax.set_ylabel(category)

    # Add percentage labels
    for i, row in cat_df.reset_index(drop=True).iterrows():
        ax.text(
            row['delay_pct'] + 0.5,
            i,
            f'{row["delay_pct"]:.1f}%',
            va='center',
            fontsize=9
        )

plt.show()

```



In [34]: `compute_delay_pct_by_category('Customer Segment')`

Out[34]:

	Customer Segment	total_orders	late_orders	delay_pct
2	Home Office	30817	17060	55.359055
0	Consumer	89420	48855	54.635428
1	Corporate	52528	28608	54.462382

Root cause analysis

```
In [35]: def top_drivers_for_region(region):
df_region = df[df['Order Region'] == region].copy()

drivers = ['Shipping Mode', 'Customer Segment', 'Department Name', 'Type', '

all_factors = []
for factor in drivers:
    temp = (
        df_region.groupby(factor)
        .agg(
            total_orders=('Delay', 'count'),
            late_orders=('Is_Delayed', 'sum'),
            avg_delay=('Delay', 'mean')
        )
        .reset_index()
    )

    temp['delay_pct'] = temp['late_orders'] / temp['total_orders'] * 100
    temp['Driver'] = factor
    temp['Factor_Level'] = factor + " : " + temp[factor].astype(str)

    all_factors.append(temp[['Driver', 'Factor_Level', 'delay_pct', 'avg_del
# Combine all drivers
final_df = pd.concat(all_factors)

# Top 10 drivers
top_factors = final_df.sort_values('delay_pct', ascending=False).head(10)
plt.figure()
```

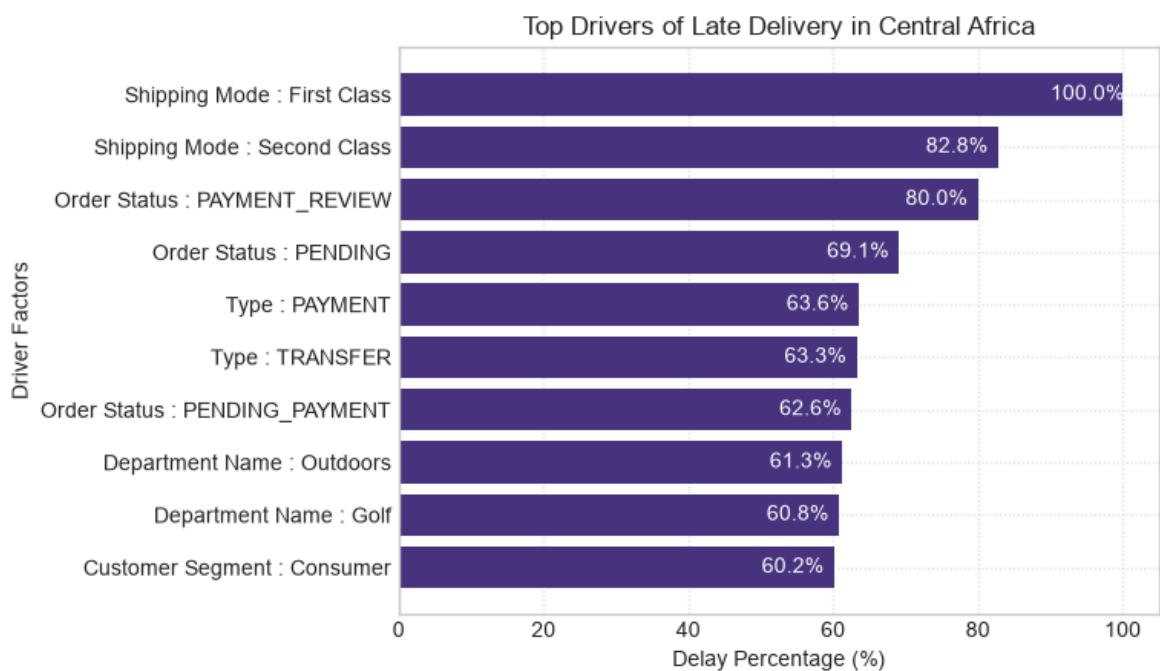
```

bars = plt.barh(top_factors['Factor_Level'], top_factors['delay_pct'])

plt.xlabel("Delay Percentage (%)")
plt.ylabel("Driver Factors")
plt.title(f"Top Drivers of Late Delivery in {region}")
plt.grid(True, linestyle=':', alpha=0.5)
plt.gca().invert_yaxis()
for bar in bars:
    width = bar.get_width()
    plt.text(width - 10, bar.get_y() + bar.get_height()/2,
             f"{width:.1f}%",
             va='center', fontsize = 10, color = 'white')

plt.show()
top_drivers_for_region('Central Africa')

```



Time Based Analysis

In [36]: df

Out[36]:

	Type	Days for shipping (real)	Days for shipment (scheduled)	Sales per customer	Delivery Status	Late_delivery_risk	Cate
0	DEBIT	3	4	314.640015	Advance shipping	0	Sp
1	TRANSFER	5	4	311.359985	Late delivery	1	Sp
2	CASH	4	4	309.720001	Shipping on time	0	Sp
3	DEBIT	3	4	304.809998	Advance shipping	0	Sp
4	PAYMENT	2	4	298.250000	Advance shipping	0	Sp
...	...	...	...	...	...	...	...
180514	CASH	4	4	399.980011	Shipping on time	0	Fi
180515	DEBIT	3	2	395.980011	Late delivery	1	Fi
180516	TRANSFER	5	4	391.980011	Late delivery	1	Fi
180517	PAYMENT	3	4	387.980011	Advance shipping	0	Fi
180518	PAYMENT	4	4	383.980011	Shipping on time	0	Fi

172765 rows × 27 columns

In [37]: `df['shipping date (DateOrders)'].max()`Out[37]: `Timestamp('2018-02-06 22:14:00')`In [38]: `# Delay % by Month, Day of Week, Hour`

```

delay_by_month = (
    df.groupby('order_month')['Is_Delayed']
      .mean()
      .reset_index()
)
delay_by_month['delay_pct'] = delay_by_month['Is_Delayed'] * 100

delay_by_day = (
    df.groupby('order_day')['Is_Delayed']
      .mean()
      .reset_index()
)
delay_by_day['delay_pct'] = delay_by_day['Is_Delayed'] * 100

delay_by_hour = (

```

```
df.groupby('order_hour')['Is_Delayed']  
  .mean()  
  .reset_index()  
)  
delay_by_hour['delay_pct'] = delay_by_hour['Is_Delayed'] * 100
```

In [39]: delay_by_month

Out[39]:

	order_month	Is_Delayed	delay_pct
--	-------------	------------	-----------

0	1	0.542542	54.254150
1	2	0.546046	54.604550
2	3	0.548094	54.809390
3	4	0.545042	54.504230
4	5	0.549980	54.998022
5	6	0.547645	54.764535
6	7	0.537310	53.731048
7	8	0.553871	55.387128
8	9	0.553885	55.388454
9	10	0.540970	54.097044
10	11	0.548609	54.860878
11	12	0.552240	55.224003

In [40]: delay_by_day

Out[40]:

	order_day	Is_Delayed	delay_pct
--	-----------	------------	-----------

0	Friday	0.544884	54.488442
1	Monday	0.555338	55.533838
2	Saturday	0.542619	54.261937
3	Sunday	0.551524	55.152418
4	Thursday	0.547202	54.720188
5	Tuesday	0.539841	53.984137
6	Wednesday	0.548452	54.845175

In [41]: delay_by_hour

Out[41]:

	order_hour	Is_Delayed	delay_pct
0	0	0.540686	54.068641
1	1	0.554227	55.422680
2	2	0.551937	55.193735
3	3	0.541420	54.142012
4	4	0.536747	53.674683
5	5	0.552672	55.267154
6	6	0.528388	52.838802
7	7	0.549882	54.988239
8	8	0.531329	53.132903
9	9	0.543575	54.357480
10	10	0.548847	54.884668
11	11	0.533742	53.374233
12	12	0.566527	56.652661
13	13	0.558672	55.867241
14	14	0.559994	55.999441
15	15	0.555914	55.591413
16	16	0.536262	53.626220
17	17	0.548392	54.839155
18	18	0.556370	55.637018
19	19	0.518847	51.884701
20	20	0.571329	57.132896
21	21	0.542390	54.238962
22	22	0.554455	55.445545
23	23	0.547986	54.798635

In [42]:

```
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(18, 6))

# Subplot 1: Delay % Trend Over Month
ax1.plot(delay_by_month['order_month'], delay_by_month['delay_pct'], marker='o',
ax1.set_xticks(range(1, 13))
ax1.set_xticklabels(['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Se
ax1.set_xlabel("Month")
ax1.set_ylabel("Delay Percentage (%)")
ax1.set_title("Delay % Trend Over Month")
ax1.grid(True, linestyle=':', alpha=0.5)

# Annotate top 3 highest
top3_month = delay_by_month.nlargest(3, 'delay_pct')
for _, row in top3_month.iterrows():
```

```

ax1.annotate(f"{row['delay_pct']:.1f}%", (row['order_month'], row['delay_pct']
            textcoords="offset points", xytext=(0, 10), ha='center', fontsi

# Subplot 2: Delay % by Day of Week
day_order = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday',
delay_by_day['order_day'] = pd.Categorical(delay_by_day['order_day'], categories
delay_by_day = delay_by_day.sort_values('order_day')

ax2.bar(delay_by_day['order_day'], delay_by_day['delay_pct'], color=primary_color
ax2.set_xticklabels(delay_by_day['order_day'], rotation=45)
ax2.set_xlabel("Day of Week")
ax2.set_ylabel("Delay Percentage (%)")
ax2.set_title("Delay % by Day of Week")
ax2.grid(True, linestyle=':', alpha=0.5)

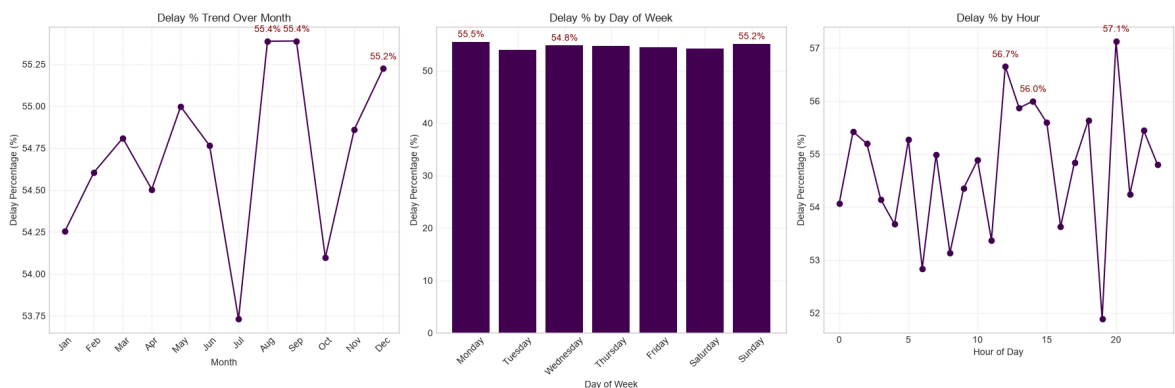
# Annotate top 3 highest bars
top3_day = delay_by_day.nlargest(3, 'delay_pct')
for _, row in top3_day.iterrows():
    height = row['delay_pct']
    ax2.text(row['order_day'], height + 0.5, f'{height:.1f}%', ha='center', va='

# Subplot 3: Delay % by Hour
ax3.plot(delay_by_hour['order_hour'], delay_by_hour['delay_pct'], marker='o', color=primary_color
ax3.set_xlabel("Hour of Day")
ax3.set_ylabel("Delay Percentage (%)")
ax3.set_title("Delay % by Hour")
ax3.grid(True, linestyle=':', alpha=0.5)

# Annotate top 3 highest
top3_hour = delay_by_hour.nlargest(3, 'delay_pct')
for _, row in top3_hour.iterrows():
    ax3.annotate(f"{row['delay_pct']:.1f}%", (row['order_hour'], row['delay_pct']
            textcoords="offset points", xytext=(0, 10), ha='center', fontsi

plt.tight_layout()
plt.show()

```



```

In [43]: from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, classification_report
from sklearn.ensemble import RandomForestClassifier
from collections import Counter
from imblearn.over_sampling import SMOTE

```

```
df.columns
```

```
In [51]: x= df[['Type','Days for shipment (scheduled)', 'Category Name','Customer Segment']
y= df['Late_delivery_risk']
```

```
In [52]: df.columns
```

```
Out[52]: Index(['Type', 'Days for shipping (real)', 'Days for shipment (scheduled)',
               'Sales per customer', 'Delivery Status', 'Late_delivery_risk',
               'Category Name', 'Customer Country', 'Customer Segment',
               'Department Name', 'order date (DateOrders)', 'Order Item Profit Ratio',
               'Sales', 'Order Profit Per Order', 'Order Region', 'Order Status',
               'Product Name', 'Product Price', 'shipping date (DateOrders)',
               'Shipping Mode', 'Order_Processing_time', 'Delay', 'Is_Delayed',
               'order_month', 'order_day', 'order_hour', 'Profitability Flag'],
              dtype='str')
```

```
In [54]: cat_cols = x.select_dtypes(include=['object', 'category']).columns.tolist()
print('Categorical columns:', cat_cols)

# Frequency encoding (Low-dimensional and robust for high-cardinality)
for col in cat_cols:
    freq = x[col].value_counts(normalize=True)
    x[f'{col}_freq'] = x[col].map(freq)

# Keep numeric columns + new encoded features, drop original string categories
x_encoded = x.drop(columns=cat_cols)
print('Shape after freq+target encoding:', x_encoded.shape)

# use encoded features for modeling
x = x_encoded

# train/test split after encoding
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42, stratify=y
)
```

```
Categorical columns: ['Type', 'Category Name', 'Customer Segment', 'Department Name', 'Order Region', 'Shipping Mode']
Shape after freq+target encoding: (172765, 9)
```

```
In [56]: x_encoded
```

Out[56]:

	Days for shipment (scheduled)	order_month	order_hour	Type_freq	Category Name_freq	Customer Segment_freq	I
0	4	1	22	0.401094	0.001945	0.517582	
1	4	1	12	0.243851	0.001945	0.517582	
2	4	1	12	0.113542	0.001945	0.517582	
3	4	1	11	0.401094	0.001945	0.178375	
4	4	1	11	0.241513	0.001945	0.304043	
...	...	...	...	...	...	...	
180514	4	1	3	0.113542	0.096055	0.178375	
180515	2	1	1	0.401094	0.096055	0.304043	
180516	4	1	21	0.243851	0.096055	0.304043	
180517	4	1	20	0.241513	0.096055	0.517582	
180518	4	1	18	0.241513	0.096055	0.517582	

172765 rows × 9 columns

In [57]: `y.value_counts()`

Out[57]:

```
Late_delivery_risk
1    98977
0    73788
Name: count, dtype: int64
```

```
In [60]: #Balancing the training data using SMOTE
print("Before balancing (train):", Counter(y_train))

smote = SMOTE(random_state=42)
x_train_bal, y_train_bal = smote.fit_resample(x_train, y_train)

print("After balancing (train):", Counter(y_train_bal))
```

Before balancing (train): Counter({1: 79182, 0: 59030})

After balancing (train): Counter({0: 79182, 1: 79182})

```
In [61]: def evaluate_model(y_true, y_pred, model_name):
print(f"\n--- {model_name} ---")
print("Accuracy:", round(accuracy_score(y_true, y_pred), 2))
print("Precision:", round(precision_score(y_true, y_pred), 2))
print("Recall:", round(recall_score(y_true, y_pred), 2))
print("\nClassification Report:\n", classification_report(y_true, y_pred))
```

```
In [63]: # Fit the Random Forest model on balanced data
rf_model_balanced = RandomForestClassifier(random_state=42)
rf_model_balanced.fit(x_train_bal, y_train_bal)

y_pred_rf_balanced = rf_model_balanced.predict(x_test)

evaluate_model(y_test, y_pred_rf_balanced, "Random Forest Classifier")
```

--- Random Forest Classifier ---
Accuracy: 0.74
Precision: 0.79
Recall: 0.75

Classification Report:

	precision	recall	f1-score	support
0	0.68	0.73	0.70	14758
1	0.79	0.75	0.77	19795
accuracy			0.74	34553
macro avg	0.73	0.74	0.73	34553
weighted avg	0.74	0.74	0.74	34553

In []: